MiniDecaf Report: Stage 5

Author: 罗兰 Ruslan Bayramov (2021080066)

Introduction

This report provides a detailed overview of Stage 5 in the MiniDecaf compiler project, which focuses on implementing function-related constructs, including function definitions, parameters, and function calls. This stage is crucial for extending the compiler's capability to handle more complex programming constructs.

Objectives

1. **Function Implementation**: Introduce and support function definitions, parameters, and function calls.
2. **Semantic Analysis Enhancement**: Extend semantic analysis to handle function-related constructs.
3. **Intermediate Code Generation**: Develop logic for generating Three-Address Code (TAC) for functions and function calls.
4. **Backend Adaptation**: Update the backend to support the newly implemented function constructs, including parameter handling and register management during function calls.

Detailed Function Implementation

Frontend Enhancements

- **AST Node Development**:
  - Add `Function`, `Parameter`, and `Call` nodes in `frontend/ast/tree.py` to represent function definitions, parameters, and function calls, respectively.
  - Provide default visitor functions for these nodes in `frontend/ast/visitor.py`.

- Define the lexical and syntax rules for these constructs, following the procedures established in previous stages.

Middle-end Logic

1. **Semantic Analysis**:
   - In `Namer` and `Typer`, modify the `visitProgram` function to traverse all functions instead of only the `main` function.
   - Implement `visitFunction` to first create a `FuncSymbol` and a function scope (`Scope(ScopeKind.FORMAL)`), which includes both internal variables and function parameters. Parameters are scanned and symbols are created for them before visiting the function body.
2. **Intermediate Code Generation**:
   - Add `PARAM/CALL` instructions in `utils/tac/tacinstr.py` for setting function call parameters and making function calls.
   - In `TACGen`, transform each function body into TAC, and implement `visitCall` to process call parameters and set return values using newly created temporary variables.

Backend Adjustments

- **Function Call Handling**:
  - Implement `visitParam` and `visitDirectCall` methods in `backend/riscv/riscvasmemitter.py` for saving function call parameters and managing function calls.
  - Handle the saving of `Temp` variables to the stack and retrieving them, particularly for parameters.
  - Manage `ra` register and callee-save registers, ensuring their preservation and restoration.
  - Design the stack layout for parameters and consider storing all parameters on the stack.
  - Implement caller-save register preservation and restoration around function calls, possibly through new pseudo-instructions during the register allocation

phase.

Challenges and Solutions

Function Implementation

- **Challenge**: Integrating function definitions, parameters, and calls into the existing compiler structure.
- **Solution**: Create new AST nodes and visitor functions for function-related constructs and extend the frontend and middle-end to handle these additions. Backend modifications are required for stack and register management during function calls.

Conclusion

In Stage 5 of the MiniDecaf project, we have successfully incorporated function definitions, parameters, and calls into the compiler. This implementation required significant enhancements in all compiler phases, from frontend AST node creation to backend register management. The addition of these function constructs significantly broadens the compiler's capabilities, allowing it to handle more complex and functional programming scenarios. This stage lays a crucial foundation for further expanding the compiler's functionality in subsequent stages.

思考题

Intermediate Representation (IR) Design for Function Calls

When considering IR design for function calls, there are two approaches:

1. **Integrated Function Call Instruction**: Combines argument passing and the function call into a single instruction.
    - **Advantages**: Simplifies IR, ensures atomicity.
    - **Disadvantages**: Less flexible, potentially complex implementation.

2. **Separated Approach for Passing Arguments and Calling**: Keeps argument passing and function calling as separate instructions.
    - **Advantages**: Greater flexibility, modular design.
    - **Disadvantages**: More complex IR, lacks atomicity.

Callee-saved vs. Caller-saved Registers in RISC-V

The standard calling convention in RISC-V includes both callee-saved and caller-saved registers for optimizing register usage:

- **Callee-saved Registers**: Saved and restored by the function being called. Ideal for preserving values needed later by the caller.
- **Caller-saved Registers**: Saved by the caller if needed after the function call. Suitable for temporary values.

The `ra` (return address) register is caller-saved because:

- The caller manages the control flow and decides the need to save the return address.
- This approach simplifies function calls by not burdening every function with saving the `ra` register, especially when multiple calls are involved.